# ASYMPTOTIC ANALYSIS OF STACK OPERATIONS: PERFORMANCE EVALUATION USING BIG-O, BIG-THETA, AND BIG-OMEGA NOTATIONS

## CSA0620- DESIGN AND ANALYSIS OF ALGORITHM

**Guided By:**
**Dr .A . SAIRAM**

**Submitted By:**
**U . Udhayashree**
**192521448**

# Problem Statement

Efficient data processing requires fast insertion and deletion operations. The stack data structure performs these operations using the **LIFO (Last In, First Out)** principle. This presentation analyzes the efficiency of **push** and **pop** operations using **Big-O, Big-Theta, and Big-Omega** notations to determine their computational complexity.

# ALGORITHM OVERVIEW

- A stack supports two basic operations:
- **Push:** Inserts an element at the top of the stack.
- **Pop:** Removes the top element from the stack.
- Since both operations access only the top of the stack, they execute in constant time without traversing other elements.

# Algorithm / Pseudocode

ENGINEER TO EXCEL

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

**Push Operation**

- Algorithm PUSH(Stack, Item)

1. If TOP = MAX - 1
   Print "Stack Overflow"
2. Else
   TOP ← TOP + 1
   Stack[TOP] ← Item
3. End

**Pop Operation**

- Algorithm POP(Stack)

1. If TOP = -1
   Print "Stack Underflow"
2. Else
   Item ← Stack[TOP]
   TOP ← TOP - 1
   Return Item
3. End

# WORKING EXAMPLE (DRY RUN)

- Initial Stack:

  []

- Push(10)

  [10]

- Push(20)

  [10,20]

- Push(30)

  [10,20,30]

- Pop()

  Removed → 30

  Stack:

  [10,20]

# COMPLEXITY ANALYSIS

| Operation | Big-O | Big-Theta | Big-Omega |
|---|---|---|---|
| Push | $O(1)$ | $\Theta(1)$ | $\Omega(1)$ |
| Pop | $O(1)$ | $\Theta(1)$ | $\Omega(1)$ |
| Peek | $O(1)$ | $\Theta(1)$ | $\Omega(1)$ |

**OBSERVATION:**

•**Big-O:** Worst-case time is constant.
•**Big-Theta:** Average-case time is constant.
•**Big-Omega:** Best-case time is constant.
Therefore, stack operations have constant-time complexity.

# COMPARATIVE ANALYSIS

| Data Structure | Insert | Delete | Access |
|---|---|---|---|
| Stack | **O(1)** | **O(1)** | O(1) (Top only) |
| Queue | O(1) | O(1) | O(1) (Front/Rear) |
| Array | O(1) (End) | O(n) | O(1) |
| Linked List | O(1) | O(1) | O(n) |

## RESULT:

Stacks provide efficient insertion and deletion but allow access only to the top element.

# APPLICATIONS

- Function call management
- Recursion
- Undo/Redo operations
- Expression evaluation
- Parentheses matching
- Browser history navigation
- Backtracking algorithms
- Depth-First Search (DFS)

# RESULTS & DISCUSSION

## Results

- Push and pop operations require **constant execution time**.
- Performance remains unchanged as the stack size increases.
- Asymptotic analysis confirms **O(1), Θ(1), and Ω(1)** complexity.

## Discussion

- The analysis demonstrates that stack operations are highly efficient because they manipulate only the top element. This makes stacks suitable for applications requiring fast insertion and deletion.

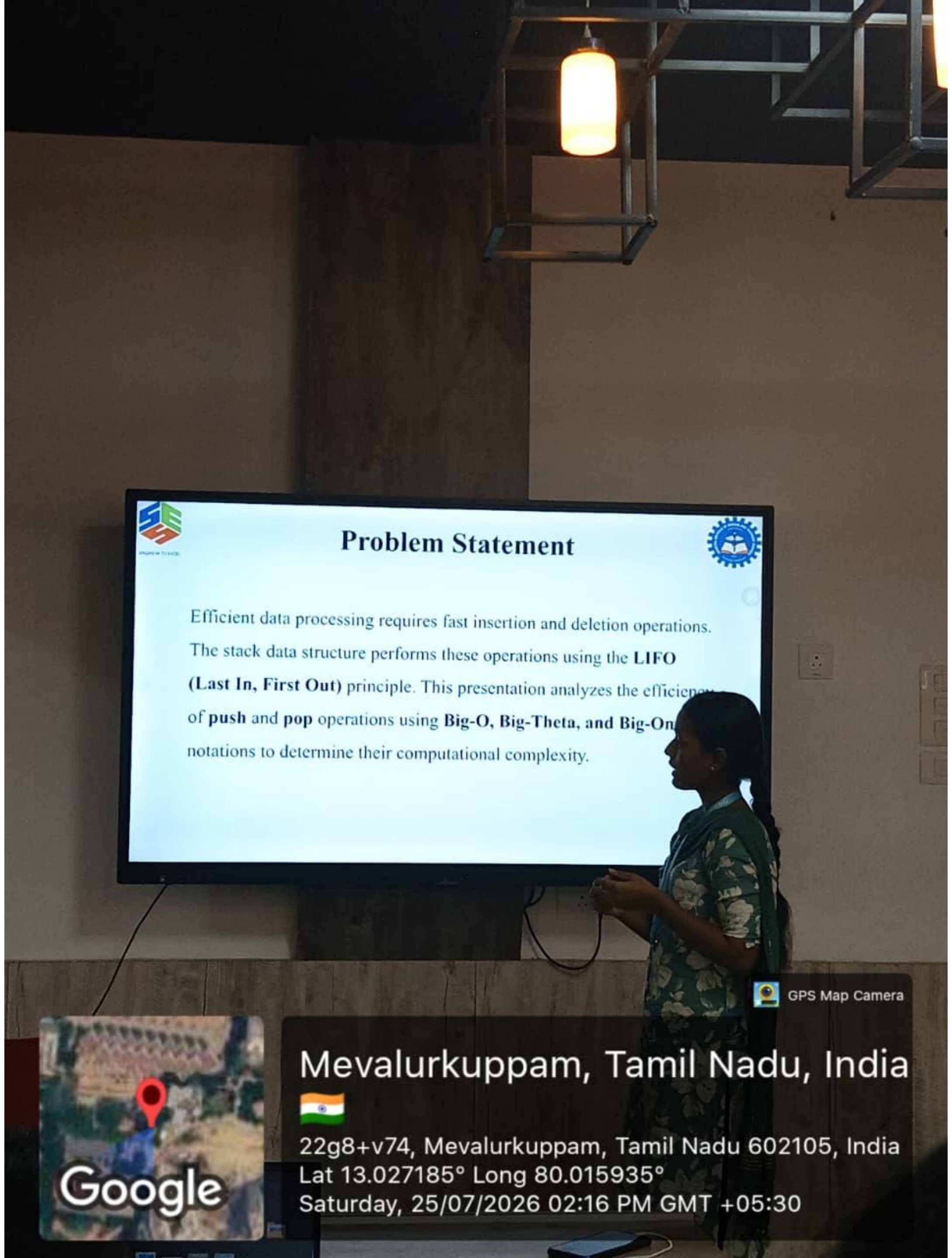
# CONCLUSION

Stack operations are among the most efficient data structure operations due to their constant-time performance. Asymptotic analysis using **Big-O, Big-Theta, and Big-Omega** proves that push and pop operations execute in **O(1)** time regardless of the number of elements. Hence, stacks are widely used in modern software systems where speed and simplicity are essential.

# REFERENCES


1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, 4th Edition, Pearson, 2023.
3. Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press, 2021.
4. Robert Sedgewick and Kevin Wayne, *Algorithms*, 4th Edition, Addison-Wesley, 2022.
5. Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, *Data Structures and Algorithms*, Pearson, 2021.

Thank You..


Problem Statement
Efficient data processing requires fast insertion and deletion operations. The stack data structure performs these operations using the LIFO (Last In, First Out) principle. This presentation analyzes the efficien
of push and pop operations using Big-O, Big-Theta, and Big-Om
notations to determine their computational complexity.
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
22g8+v74, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.027185° Long 80.015935°
Saturday, 25/07/2026 02:16 PM GMT +05:30
Google